

Tampere University
COMP.SE.610
COMP.SE.620

Unit of Computing Sciences
Software Engineering Project 1
Software Engineering Project 2

G05
Snowledge

Test Report

Saku Hakamäki
Eero Kainulainen
Toivo Knuutinen
Kaapo Kontinen
Eetu Koskela
Huyen Pham
Eemeli Ruohomäki

Version history

Version	Date	Author	Description
1.0	07.12.2025	All members of G05	Add chapters significant for delivering document to Mindhive. Includes many WIP chapters, which are mentioned as such.
2.0	11.12.2025	Saku Hakamäki Eetu Koskela	Ch. 1.4 Ch. 4.1 Ch. 4.4 Ch. 5

Contents

1	Introduction	4
1.1	Purpose and scope of document	4
1.2	Product and environment	4
1.3	Project constraints related to testing	4
1.4	Definitions, abbreviations and acronyms	5
2	Testing process	6
2.1	General approach	6
2.2	Testing roles	7
2.3	Test schedule	7
2.4	Test documentation	8
3	Testing Tools	9
3.1	Web testing tools	9
3.2	Mobile testing tools	10
4	Test cases and results	10
4.1	Test results	10
4.2	Unit testing	11
4.2.1	Web unit testing	11
4.2.2	Mobile unit testing	12
4.3	Integration, system/E2E testing	13
4.4	Acceptance testing	13
4.5	CI/CD Workflows	14
4.5.1	High-level overview	14
4.5.2	Alternative workflows	15
4.5.3	Challenges and Complexity	16
5	Bugs & Issues	16
	APPENDIX A: Table of Manual Test Cases	17
	APPENDIX B: Table of Found Bugs and Issues	18
	APPENDIX C: Acceptance Testing Results and Response	19

1 Introduction

1.1 Purpose and scope of document

The purpose of this document is to showcase the outcomes of the planned testing processes for the Snowledge application (fi: Lumisovellus). This Test Report document answers the following main questions: what was tested, how and how much it was tested, who tested it, when the testing was done, what kind of test data was used, and what were the results of the testing overall.

Due to the nature of the project, being a full rewrite of the existing application by following a reimagined North Star vision, also the testing processes have been purpose built from ground-up. The objective has been to create tests, which would ensure the functionality and support quality of the application also in the future. In this project work, the testing suite consists of unit, integration, system, end-to-end (E2E), and acceptance tests.

1.2 Product and environment

The product under development in this project is the Snowledge web and mobile application, as already above mentioned. The main features of the Snowledge application are rescue functionality and representing snow conditions using the map and the segments in it. Due to project scope changes mentioned already in the Project Plan document, the focus of this project work has been mainly on recreating the web and mobile frontends according to the North Star vision. Another project group, later referred to as G26, has been working on the backend implementation, in addition to developing an integration to the Suunto sports watches.

Since the development of the frontends and the backend have been separated into different project groups, it is increasingly important to test these components to a high standard. Testing them was done individually, but also together with the integrated frontend and backend. The main responsibility for testing the web and mobile frontends were on this project work, being G05, while the focus of tests that G26 performed were on the backend and their Suunto integration. It is a shared responsibility between the two groups that these components work together as planned.

1.3 Project constraints related to testing

The testing related project constraints were limited to the map data provider Mapbox and emergency calls. When the testing was performed, it was ensured that the Mapbox API limits were not exceeded. Crossing the API limits would have resulted in unwanted costs for the customer. Also, when the rescue functionality was tested, actual emergency calls were not made.

1.4 Definitions, abbreviations and acronyms

Table 1. Definitions, abbreviations, and acronyms.

Term	Definition
Acceptance Test	Customer-driven testing ensuring the product meets expectations
API	Application Programming Interface; interface enabling communication between frontend and backend
Backend	Server-side logic, data processing, and APIs
CD	Continuous Deployment/Delivery; automated process of deploying validated code
CI	Continuous Integration; automated process of validating code when changes are committed
CI/CD	Combined Continuous Integration and Continuous Deployment pipeline
Codecov	Coverage reporting service used in CI
Coverage	Percentage of source code executed by automated tests
Corepack	Node tool wrapper used to manage package managers like pnpm
DOM	Document Object Model; browser-level API used heavily in web testing
Drift	Database and persistence library used for offline caching in the mobile app
E2E	End-to-End testing; tests full workflows across the entire system
Flutter	Cross-platform UI framework used for the mobile application
Frontend	Client-side application (web or mobile) interacting with users
G05	Project group responsible for the Snowledge frontend implementations
G26	Project group responsible for backend implementation and Suunto integration
GitHub Actions	Automation platform used for CI/CD workflows
Gitleaks	Tool used to detect secrets and sensitive data in the codebase
HTML	HyperText Markup Language, used for web structure and test reports
ID	Identifier used in test case tables, database rows, or flows
Integration Test	Test verifying that multiple components work correctly together
jsdom	JavaScript-based DOM emulator used for Vitest component testing
JSON	JavaScript Object Notation; used for configs and API communication
LCOV	Code coverage format produced by Flutter and web tests
Mapbox	Mapping platform providing map tiles and geospatial functionality
Mockito	Mocking framework used for Flutter/Dart testing
MySQL	SQL database system used by the backend
Next.js	React-based web framework used for the Snowledge web application
Node.js	JavaScript runtime used to execute tests and build web app

North Star	Name of the document/instructions describing how the large-scale rewrite could be done for the legacy application.
npm	Node Package Manager; used for installing dependencies
OAuth	Industry-standard authentication protocol for secure login
OpenAPI	Standard for generating backend API clients
Playwright	Browser automation tool used for E2E testing on the web
PR	Pull Request; code change proposal reviewed before merging
Prisma	ORM used to sync schemas during CI backend tests
QA	Quality Assurance; processes ensuring software meets quality standards
React	JavaScript library used for building web interfaces
Rollup	JavaScript bundler used in the frontend
Snower	Hardware device providing snow measurement data
Suunto	Sports watch ecosystem
System Test	Test evaluating the behaviour of the entire system
Tavu.io	Customer's hosting provider
Turbo / Turborepo	Build system used for monorepo optimization
UI	User Interface
Unit Test	Test verifying a single component or function in isolation
Vitest	Unit test runner used for web testing

2 Testing process

The overall testing process consisted of both manual and automated testing. The quality assessment was done separately for the web and mobile frontends, and together as integrated parts to the backend.

Coding conventions and quality guidelines were enforced using automated linting with the GitHub Actions CI/CD workflows. The linting process was planned to be divided into two stages, being fast and heavy lint. Due to time constraints and prioritization, heavy linting was never implemented.

The following subchapters discuss the testing process from different aspects. Initially, the process is described in general. Additionally, the test results are summarized, and the different types of testing conducted to the Snowledge application are showcased.

2.1 General approach

The general strategy for testing was carried out using multiple different types of tests. Unit tests were used to ensure that individual components of the application work as intended. Integration and system tests verify that these individual components as well as the whole application work as expected. Additionally, acceptance testing was performed to ensure that the implementation quality corresponds to the expectations that

the customer and the end users have set for the whole application. The acceptance testing was performed with the help of the customer, who also acted as end-users.

The focus in testing was on integration, and system/E2E tests. Different kinds of analytic assessments, such as evaluations of usability, performance and reliability, were conducted as supplementary activities. These assessments were not the main focus of the testing process and were performed primarily through qualitative methods and user feedback rather than formal functional testing or quantitative metrics. Nevertheless, any feedback regarding these aspects were considered and noted for future use.

As already above briefly mentioned, the CI/CD pipeline, implemented using GitHub Actions, was created to automate the quality assurance process for the application, and running the created tests was part of it. The use of the GitHub Actions is described in more detail in one of the following subchapters dedicated to it.

Some bugs and issues were found during the testing of the application. Those bugs were noted, collected to a single table. Further information on bugs and issues can be found in a later chapter.

2.2 Testing roles

The different testing responsibilities were divided into separate testing roles. Individual developers were responsible for implementing the test cases while the whole project group was responsible for getting the test cases to pass and the application corresponds to the expected behaviour. Carrying out the manual test cases listed in the Excel test case table was the responsibility of the PO. The customer was involved in acceptance testing, acting as future end-users. The idea and reasoning behind these future end-users were that they were not involved in the main development process and they could provide an outsider kind of view to the application. In other words, they did not know what was happening under the hood, and they treated the application as a black box.

The backend of the application was developed by the other project group, G26. They are mainly responsible for testing the backend. However, in the end the overall responsibility of the functionality of the application is on both project groups.

2.3 Test schedule

The testing schedule was broken down into the following stages: unit testing, CI/CD pipeline, integration testing, system and E2E testing, bug fixing and re-testing, and finally result reporting.

It was planned to have the CI/CD pipeline completed as soon as possible, preferably in the end of Sprint 3. Nevertheless, due to being unexpectedly laborious part of the project and various challenges during implementation, getting it up and working it took longer than expected, and it was finalized in the end of Sprint 5. After finalization, it took charge over the responsibilities defined for it.

The integration testing started immediately when the actual integration of the frontend and backend began. Additionally, system and E2E testing began in the QA Sprint. These tests were conducted as was planned.

The difficulties in estimating the time required for bug fixing and retesting before the bugs had actually appeared were noted when planning the tests. A rough estimate of one week was given, but approximately half a week was used for them. This was mainly due to the deployment of the testing applications taking longer than expected.

The test versions of the applications were deployed to the platforms planned for them. The web applications test version was available running on the Tavu.io virtual machine next to the pre-North Star application at <https://beta.lumisovellus.fi>, while the mobile applications were deployed to Google Firebase. As noted above, doing this took longer than expected. The mobile side difficulties were related to deploying the iOS application, as it had some extra steps when compared to the Android counterpart. For web and backend, challenges were faced due to the resource limitations of the virtual machine.

Due to a possible meeting with the customer and Mindhive contact on the final project week, the delivery of the project documents and source code was earlier than expected. This understandably limited the time to do the testing, deployment, and documentation. The effect of this was more on the quantity of testing rather than quality. Additionally, during the QA Sprint the project team was running quite low on working hours to spent, so the hour usage was optimized to avoid going overbudget as much as possible.

2.4 Test documentation

The tests performed were documented using manual and automatic methods. The test frameworks used in the web and mobile applications provided automatically generated reports. In the case of the web app, the framework generates a full test report which contains step-by-step results and detailed information in case of failures. In the case of the mobile application the tool provides coverage reports as well as reports from executed tests. The CI/CD pipeline collects test result artifacts from the workflow.

The results from manual testing were reported using an Excel table. These are all integration and system testing test cases. The Excel table can be found from the appendices, and it contains the following columns:

- Test Level
- Platform
- ID
- Name
- Area
- Description
- Preconditions
- Test Steps
- Expected Outputs
- Actual Result
- Status
- Comment

The found bugs and open issues are also documented in a single place for easier utilization in the future. More detailed description of them can be found from the dedicated chapter for them further below.

3 Testing Tools

The tools used for testing were the same ones that were planned in the Test Plan document. As a reminder, Playwright and Vitest are used for the web application, while Flutter's built-in tools are used for the mobile application. The following chapters describe the statistics for the web and mobile tests, respectively.

The CI/CD pipeline has been constructed with GitHub Action as earlier mentioned, due to being naturally functional and integrated with the chosen version control platform GitHub. A tool within GitHub Actions that is worth mentioning here is Gitleaks. It is used to detect and prevent security vulnerabilities in the code base. The use of GitHub Actions is further described in more detail in the following chapters.

Some deployment platforms were used to test the developed versions of the Snowledge web and mobile applications. Namely, Google Firebase was the chosen tool for deploying the Android and iOS applications, while the web application was deployed with the new backend on the Tavu.io virtual machine next to the pre-North Star application. Further documentation on the deployment process can be found in Markdown files contained in the GitHub repository.

The deployed application backend uses real customer data extracted from the legacy application, as well as other realistic test data. The real data consists of the segments and snow data types. Realistic test data, such as guide updates, snow observations, and user accounts, were seeded into the backend database using a script provided as part of the backend codebase. This same dataset was also used during integration testing and throughout the manual testing phase.

The following subchapters highlight the tools used for web and mobile testing, respectively.

3.1 Web testing tools

The tooling for testing the web application were Vitest and Playwright, as was planned. We use these tools to perform a combination of unit tests, component tests and end-to-end tests to ensure code quality, prevent regressions and validate user interactions. The test suite can be ran easily using the Node command.

All unit tests and component tests are executed with Vitest, configured with jsdom environment for DOM manipulation. The tests use the React testing libraries for component rendering, interaction testing and simulating user interactions. Mocking is handled through Vitest's build-in module, allowing isolation of components and logic from external dependencies like API calls and internationalization. When executed, Vitest produces both default console output and an HTML report.

Playwright tests validate full user workflows across multiple browsers (Chromium, Firefox and Webkit). Playwright is configured for automatic HTML reporting and trace collection on test failures for debugging.

Key statistics logged during test execution include total tests run, pass/fail counts, execution time per test, and failure messages with stack traces. Overall coverage varies

(68.66% coverage of all statements). Best-covered areas reflect priorities: authentication actions (100% coverage), map 3D rendering logic (100%), weather view components (100%) and critical internationalization utilities (95%), while lower coverage in data loading and such due to interaction with external APIs that are difficult to mock in isolation.

3.2 Mobile testing tools

On the mobile side, the planned and used tools were the same. All automated tests were executed with Flutter's built-in test runner (flutter test), using the flutter_test package for unit and widget tests. Mockito was used to generate mocks for API clients and services, so repositories and service logic could be tested in isolation without a running backend.

A separate integration test suite under test/integration/ used the same Flutter runner but talked to a real backend. These tests authenticated automatically (either via a provided token or by logging in / registering a test user) and exercised full rescue help flows such as creating, cancelling and completing help events.

Coverage data was collected with flutter test --coverage, which produced the coverage/lcov.info file. After converting it to HTML with genhtml, the final run reported 517 covered lines out of 2,958 (17.5% line coverage). Most automated tests targeted the logic-heavy areas, which is reflected in the highest coverage results: map repositories (81.1%), map services (100%), map widgets (72.4%), rescue services (77.6%), rescue mappers (40.2%) and rescue repositories (100%). UI-heavy screens and peripheral features remained lightly covered or uncovered, as they were primarily validated through manual and exploratory testing. The Flutter runner, Mockito mocks, and LCOV reports together provided the key statistics used during development: number of tests run, pass/fail status and line coverage.

4 Test cases and results

The following chapters discuss the whole testing process and the outcomes of it in more detail. The different types of carried out tests are described in separate sections to clarify the overall process. Additionally, there is also a subchapter that explains the CI/CD pipeline.

4.1 Test results

The overall outcome of the testing process indicates that the Snowledge application is functioning as expected within the scope defined for this project, but it is not production-ready since it does not have all the functionality from the legacy application. Most critical user flows on both web and mobile platforms, such as map interaction, weather and segment browsing, authentication, and core rescue request functionality, were verified through a combination of automated tests, structured manual testing, and exploratory testing.

The manual system and integration testing were executed using the test case suite documented in the Excel table. Out of 80 planned test cases, 41 were executed and passed, with one failing, while 38 were intentionally skipped due to requirement

changes, removed features or out-of-scope rescue functionality identified earlier during implementation.

All automated unit tests for both web and mobile passed. Web testing achieved good coverage in logic-critical areas, and mobile tests thoroughly validated map- and rescue-related service logic.

Through the combination of these testing procedures, the group identified several issues, which were documented in a bug list Excel table.

Acceptance testing produced an additional set of findings reported by the customer in a Google Docs document. Many of these were usability-related, and some overlapping bugs had already been identified and fixed in a later version. All fixed items from this list were added to the bug list Excel sheet. In total, 34 issues were recorded, leaving roughly a third of them open. In addition to these, the acceptance testing results included around 15 further points marked for future development.

So, in the end, based on the executed tests and the results obtained, the application meets the expected quality level at this stage, given that the scope was never to produce a production-ready application. The implemented features function mostly as intended, but additional testing, especially non-functional and feature refining, would be needed before the software could be considered ready for real-world deployment.

4.2 Unit testing

The development and execution of unit tests was divided so as each person developing a certain application area would be responsible for creating unit tests for it. The following chapters discuss the unit tests for web and mobile applications, respectively.

4.2.1 Web unit testing

Unit testing for the web application focused on core logic, UI components, and critical utilities that drive the application. These tests were written Vitest and React Testing Library, allowing components, hooks, and utility functions to be verified in isolation from external APIs and browser-specific behaviour.

The unit test suite primarily targeted:

- Authentication logic: Actions for logging in, registering users and validating tokens were tested to ensure proper token handling and error responses
- Map 3D components: Interactive elements such as Snower monitor information panels, snow type selection comboboxes, and area selection flows were tested for correct rendering, user interaction callbacks, and state management
- Weather components: The weather view component was tested to verify data fetching, display formatting, and error handling
- Utility functions: Snow type transformations, internationalization and map camera state persistence were validated for correct input/output behaviour
- Hooks: application critical hooks, such as use-auth, which is used for authorization is tested to confirm proper state management and role-based access-control

Component tests used the React Testing Library to render components in isolation with mocked dependencies. The Vitest vi mocking module replaced API clients, internationalization providers, and authentication context. Test simulated user interactions (clicks, form input, keyboard navigation) using the User Event Testing Library and verified that components rendered expected content, called correct callbacks and updated state appropriately.

Unit tests for utility functions and services validated input/output correctness without rendering. For example, snow type utilities were tested by passing various snow type IDs and verifying the correct translation keys were returned. Map camera state persistence tests confirmed localStorage read/write operations functioned correctly.

For mocking strategy, as mentioned above, external dependencies were isolated using Vitest's vi.mock(). API calls were mocked to return controlled test data, allowing tests to verify both success and error scenarios without requiring a running backend. Internationalization was mocked to validate translation key usage without loading full language files.

All 73 web unit tests pass. The test suite covered 68.66% of statements, with coverage concentrated in logic-heavy and critical path areas such as authentication, displaying Snower information, locale switching and error handling.

UI-heavy interactions and full map gesture handling remain lightly covered due to the difficulty of simulating complex MapBox GL events in a jsdom environment. Data loading utilities have a lower coverage as they interact directly with external APIs that are challenging to mock comprehensively. Integration tests with a real backend provide coverage for these code paths.

4.2.2 Mobile unit testing

Mobile unit testing focused on the core logic of the map and rescue features. Tests were written with flutter test and Mockito, allowing repositories, services, and mappers to be checked in isolation. The rescue tests confirmed that creating, cancelling, and completing help requests worked end-to-end at the domain–service boundary. Map-related tests validated correct API calls, error handling, and data conversion. Widget tests covered key interactive components such as the AreaCard flow, verifying that callbacks and UI state behaved as expected.

All mobile unit tests passed. In total, the unit test suite covered 464 out of 2,958 lines (15.7%), with the strongest coverage in logic-heavy areas such as map repositories (81.1%), map services (100.0%), map widgets (72.4%), rescue data mappers (40.2%), and simple domain models (27.3%). UI-heavy screens and some view models had noticeably lower automated coverage and were instead exercised mainly through manual and exploratory testing.

For the request help functionality, tests were created to check that everything works correctly from start to finish. They specifically check three things:

- Sending a help request: When the app sends a help request, the repository correctly turns the domain model into the format the service expects, and then correctly turns the service's response back into the domain version.

- Handling different types of requests and statuses: The tests try every kind of help type (health, equipment, lost) and every possible status (active, completed, cancelled) to make sure the app handles them all correctly.
- Cancelling or completing help requests: If the user cancels or completes a request, the app sends the right command to mark it as cancelled or completed.

Mockito was used to fake the service so the tests could check both the data being sent and the data being returned. All the tests passed, meaning that the app correctly sends help requests, understands the responses, handles all types and statuses, and correctly cancels or completes requests.

4.3 Integration, system/E2E testing

Integration and system testing are grouped into one here. These tests were split into 3 different categories of testing, automated, manual and exploratory.

Automated tests were written separately for various features and functionalities of both the web and mobile applications. On the web side, a total of 12 end-to-end (E2E) tests were implemented. On the mobile side, automated integration tests focused primarily on the rescue feature, comprising 14 cases. In total, the number of non-unit automated test cases amounts to 26, all of which pass.

Manual integration and system tests were documented in the test plan using an excel table. The list was used to conduct the structured part of manual testing, target of which was the main functionalities of the application. The amount of test cases was quite large, as many of them are duplicates because both web and mobile had their own test case. Due to the changes in requirements and the rescue being largely out of scope, there were quite many test cases that were skipped. The tests were conducted initially, and bugs were reported and fixed, after which those fixed were retested to be confirmed as passed. More details regarding each test case and their targets are listed in the table. Summary of the table is that there were 80 test cases out of which 41 were passed and 38 skipped due to the aforementioned reasons. One test case was noticed late into the testing and was left unfixed.

In addition to the manual structured testing, exploratory testing was also conducted. There were no real test cases written about this procedure. The main scope for this was to test the functionalities that the automated and structured manual testing did not cover. How it was conducted, was that the demo applications that had been deployed were used by multiple group members in different ways to find flaws or inconsistencies. This was quite fruitful and resulted in over 20 findings. The time spent specifically for this type of testing was a few hours, which could have been more, had we had more hours to spare.

4.4 Acceptance testing

Acceptance testing was conducted by the customer during the QA sprint. This was made possible by deploying the web application to their existing deployment server and deploying the mobile applications to Google Firebase. The customer performed their mobile testing using iOS devices.

The customer provided their feedback in a Google Docs file, which is included in the appendix. This feedback was received on the evening of the final day of the QA sprint, with the timing influenced in part by the iOS app deployment taking additional time. The document contained separate lists of bugs and issues related to usability and functionality for both the mobile and web applications. The group responded directly within the same document, addressing each point and requesting clarification when needed. Some of the issues had already been identified by the group earlier, and several had already been fixed or deployed.

By the time the document was received, the project hours were nearly exhausted. This limited the resources available for addressing the issues listed in the feedback. The issues the group chose to address were primarily cosmetic changes and other minor adjustments. Remaining issues were marked for future development and noted in the final report.

4.5 CI/CD Workflows

The continuous integration (CI) system for this project is structured around a reusable workflow referred to as the “Unified CI.” This workflow encapsulates all core validation steps required for both backend and frontend components of the monorepo. Rather than duplicating logic across multiple CI files, separate event-specific workflows delegate their work to this shared pipeline and adjust its behaviour through configuration parameters. This architecture promotes maintainability, consistency, and adaptability across different branches and development scenarios.

4.5.1 High-level overview

The primary purpose of the CI workflows is to automatically evaluate the quality, safety, and correctness of code whenever changes are introduced into the repository. By performing systematic checks at each pull request and branch push, the workflows prevent regressions, reduce integration failures, and support a stable development process.

The objectives of the workflows are multi-fold. They aim to ensure code safety by blocking insecure or malfunctioning changes from entering key branches. They also seek to optimise performance by executing only the relevant parts of the pipeline depending on which files were modified. Furthermore, the workflows maintain consistency across backend and frontend services by centralising the logic in a unified pipeline. A final objective is to provide transparency, enabling developers to observe test failures, coverage reports, and audit results directly within the pull request.

A variety of tools support these objectives. GitHub Actions provides the workflow execution environment and orchestration. Node.js tooling (including npm, Corepack, and the Turbo build system) is used extensively for dependency management, building, and testing. Backend tests run against a real MySQL service, managed through Prisma for schema creation and database synchronisation. Codecov is used to upload and visualise coverage metrics, while ESLint, Vitest/Jest, and type-checking tools enforce code quality. Security responsibilities are handled through npm-audit and audit-ci. On the frontend side, the workflow integrates Flutter, Playwright browsers (when applicable), and OpenAPI client validation.

4.5.2 Alternative workflows

Although all event-based workflows ultimately invoke the Unified CI, each one configures and triggers the pipeline in different ways, depending on the branch and intended level of strictness.

Unified CI workflow is executed only when invoked via `workflow_call`. It begins with a change-detection step that determines whether backend or frontend paths were modified. Based on this result, the workflow conditionally executes backend linting, testing, security checks, and building, or corresponding frontend validation, linting, type checking, building, testing, and smoke testing. The workflow technically includes support for both heavy linting and smoke tests, as these are triggered through specific input parameters in the calling workflows. However, neither mechanism is fully implemented at this stage. The heavy lint job invokes a workspace-level “lint:heavy” script that does not currently exist in any relevant package, meaning the job executes without performing any additional checks. Similarly, the smoke testing job contains only a placeholder command rather than actual end-to-end tests. These omissions were the result of time constraints, as available development time was directed towards implementing and stabilising the core CI functionality required for reliable pipeline execution.

Pull request to “dev” branch workflow runs whenever a pull request targets the dev branch. It invokes the Unified CI with full capabilities enabled, including building, security auditing, and smoke testing, even though the smoke test job itself is not yet implemented. The intent is to ensure that all contributions entering the shared development branch undergo comprehensive validation before merging.

Pull request to the “frontend” branch workflow activates on pull requests targeting the frontend branch. It applies stricter controls than the other PR workflows by raising the required coverage threshold to 80% and enabling heavy linting. Although heavy linting is structurally enabled, its practical effect is currently negligible because the necessary frontend script has not been implemented. The objective remains to maintain a particularly high standard for frontend development work once the feature is complete.

Master workflow is triggered by pull requests into the master branch. It mirrors the configuration used for dev, applying a full suite of checks, including builds, security verification, and smoke testing. As with the other workflows, the smoke test step runs only as a placeholder at this point, pending future implementation.

Pushing to any branch workflow runs whenever changes are pushed to any existing or new branch. It is intentionally lightweight: although heavy linting is enabled, backend full stages and security checks are disabled. Heavy linting functions only at a structural level, as the underlying functionality is not yet implemented. This allows developers to receive rapid feedback during development without incurring the full cost of the more thorough PR workflows.

The workflows differ primarily in strictness, scope, and runtime. Pull-request workflows enforce comprehensive checks, while branch-push workflows prioritise speed. The workflow for the frontend branch introduces the strictest requirements, including higher coverage thresholds and heavy linting, even though heavy linting is not yet operational. In contrast, the push-to-branch workflow reduces overhead by skipping security audits and backend build stages. These differences allow the CI system to balance thoroughness and efficiency depending on the development context.

4.5.3 Challenges and Complexity

The implementation of this CI structure introduces several challenges. Since the repository contains both backend and frontend systems, reliable change detection is essential to avoid unnecessary job execution or the accidental omission of important validations. Backend tests require coordination with a temporary MySQL instance, which can introduce timing issues and dependency concerns. Additionally, the pipeline includes workarounds like patching npm versions or installing native Rollup binaries, which adds maintenance complexity and may require revision as dependencies evolve.

The frontend validation steps also introduce their own challenges. Flutter toolchains, Playwright setups, and OpenAPI client generation must remain stable and compatible with the rest of the system. These tools can be sensitive to version changes, making long-term maintenance a slight concern. The incomplete heavy linting and smoke testing features highlight another complexity: prioritising which components of the pipeline must be made fully operational within limited development time.

Looking ahead, the CI system will require semi-regular updates to accommodate evolving dependency ecosystems. As new services or test types are added to the repository, the Unified CI may grow in complexity and require restructuring. Expanded end-to-end or smoke testing may also introduce longer execution times and potential flakiness, demanding improved test isolation and better monitoring. Finally, as dependency trees grow, security audit tools may generate increasing amounts of noise, necessitating careful configuration management to preserve meaningful output.

Overall, while the CI architecture is quite sturdy and flexible it requires ongoing upkeep to ensure reliability, efficiency, and compatibility with rapidly changing tooling ecosystems.

5 Bugs & Issues

The bugs and issues identified during the QA sprint were documented in an Excel sheet. This approach was chosen to make tracking progress, and identifying any unresolved items, as straightforward as possible. The table itself was not highly detailed, as fixes were implemented quickly and further discussion took place within the team via Discord. The table is included in the appendix.

Decisions regarding which bugs or issues to address were also discussed on Discord. As previously mentioned, each team member had already spent a considerable number of hours on the project at this stage, so only the most critical and easily resolvable issues were prioritized. Additionally, not all items listed were actual bugs; some represented unwanted functionality or areas that required optimization.

The issues the group addressed from the acceptance testing results were added to the same bug list. In summary, a total of 34 issues exist on this list. Of these, 18 were fixed, while 5 were reclassified as non-issues or mislabelled items, leaving 11 open items.

APPENDIX A: Table of Manual Test Cases

Test Level	Platform	ID	Name	Area	Description	Preconditions	Test Steps	Expected Output	Actual Result	Status	Comment
System	Mobile	MAP-001-MOBILE	Map loads correctly	Map	Verify that the map view loads successfully in the web application.		1. Navigate to the map page. 2. Wait for map to render. 3. Observe that map elements load properly	Map tiles render for pallas area; no error is shown.	Map takes around 4 seconds to load on the demo deployment	Pass	
System	Web	MAP-001-WEB	Map loads correctly	Map	Verify that the map view loads successfully in the web application.		1. Navigate to the map page. 2. Wait for map to render. 3. Observe that map elements load properly	Map tiles render for pallas area; no error is shown.	Map is loaded very fast	Pass	
System	Web	MAP-002-WEB	Map centers on correct location on load	Map	Map should always center on pallas area		1. Open the map 2. Wait for it to render 3. Observe location	Map centers on pallas ski area	Map centers on pallas area correctly	Pass	
System	Mobile	MAP-002-MOBILE	Map centers on correct location on load	Map	When location is allowed, the map should center on the user's current location on load.	Location permission granted and location on	1. Ensure app has location permission. 2. Open mobile app and go to Map. 3. Observe map initial centering.	Map centers on user's current location automatically	Map centers on users location on load after user has enabled location	Pass	
System	Mobile	MAP-003-MOBILE	Show user location marker when location allowed	Map	Verify that user's own location is visible on the map if permission is granted.	Location permission granted and location on	1. Open map on mobile with location enabled. 2. Check for user-location marker.	User-location marker/icon is shown at correct coordinates.	User's location can be seen on the map as a blue dot	Pass	
System	Mobile	MAP-004-MOBILE	Map usable offline	Map	Verify that the map is usable in offline mode using cached tiles.	User previously opened map area online on mobile (tiles cached).	1. Open the app and navigate to the map 2. Switch device to offline/airplane mode 3. Pan/zoom within previously viewed area.	Cached area loads without errors; panning/zooming within cached extent works.	User is notified that they are offline. Previously loaded map area works	Pass	
System	Mobile	MAP-005-MOBILE	Hide segments layer	Map	Verify that segments can be hidden from the map via layer control.	Map loaded; segments visible.	1. Open map. 2. Open layers/settings. 3. Toggle 'Segments' off.	All segment geometries disappear from map; base map remains.	When pressing the button in the bottom left and selecting segments, the segments can be toggled on and off	Pass	
System	Web	MAP-005-WEB	Hide segments layer	Map	Verify that segments can be hidden from the map via layer control.	Map loaded; segments visible.	1. Open map. 2. Open layers/settings. 3. Toggle 'Segments' off.	All segment geometries disappear from map; base map remains.	When pressing the button in the bottom left and selecting segments, the segments can be toggled on and off	Pass	
System	Web	MAP-006-MOBILE	Move to ski areas	Map	Verify user can use the centering quick options to ski areas	Map loaded	1. Open map 2. Open area shortcut menu 3. Select pallastunturi	Map centers on pallas ski area	When pressing the ski area listing on the bottom right corner the user can select pallas area and the map mover there	Pass	
System	Web	MAP-007-MOBILE	Move to own location	Map	Verify user can use the centering quick options to own location	Location permission granted and location on	1. Open map 2. Tap the own location marker in the corner of the map	Map centers on user's current location	When user clicks on the own location marker on the map, the map centers there correctly	Pass	
System	Mobile	LOC-001-MOBILE	Prompt for allowing location sharing	Location tracking	Verify that the application prompts the user to allow location sharing after login.	First load of the app	1. Log in on mobile. 2. Observe prompts related to location.	App displays prompt to allow location sharing with clear allow/deny options.	Location sharing permissions are queried from the user on first load	Pass	

Integration	Mobile	LOC-002-MOBILE	Location sent to backend for 72h when enabled	Location tracking	Verify that when user enables location sharing, periodic location updates are sent to backend for 72 hours.	User logged in (mobile); user accepted location sharing; backend monitor/logs available.	1. Enable location sharing in app. 2. Move device / simulate location. 3. Check backend/logs for location updates over time. 4. After 72h expiry, attempt to send location again.	Location updates are sent during the 72h window; after 72h the backend no longer receives updates for that session/user unless re-enabled.	N/A	Skip	Not implemented
System	Mobile	LOC-003-MOBILE	Disable location sharing	Location tracking	Verify that the user can disable location sharing after enabling it.	User logged in (mobile); location sharing enabled.	1. Open settings/location in app. 2. Turn off location sharing. 3. Move device / simulate location. 4. Check backend for updates.	Location sharing state changes to disabled; no more location updates are sent to backend.	N/A	Skip	Not implemented
System	Mobile	SEG-001-MOBILE	Segments load on map	Segments	Verify that segment data loads and is displayed on map.	Map loaded; network available.	1. Open map on web/mobile. 2. Zoom to an area with known segments.	Segments are rendered with correct shapes and styling.	Segments are overlaid on the map correctly	Pass	
System	Web	SEG-001-WEB	Segments load on map	Segments	Verify that segment data loads and is displayed on map.	Map loaded; network available.	1. Open map on web/mobile. 2. Zoom to an area with known segments.	Segments are rendered with correct shapes and styling.	Segments are overlaid on the map correctly	Pass	
System	Mobile	SEG-002-MOBILE	Segments interactable and show information	Segments	Verify that tapping/clicking a segment opens its information view.	Segments visible on map.	1. Select a segment on map. 2. Observe info panel.	Segment is highlighted; information view shows segment name/id and other observation related fields.	Segments can be selected to showcase information about them. They are highlighted when this is done	Pass	
System	Web	SEG-002-WEB	Segments interactable and show information	Segments	Verify that tapping/clicking a segment opens its information view.	Segments visible on map.	1. Select a segment on map. 2. Observe info panel.	Segment is highlighted; information view shows segment name/id and other observation related fields.	Segments can be selected to showcase information about them. They are highlighted when this is done	Pass	
System	Mobile	SEG-003-MOBILE	Segment filtering	Segments	Verify that user can filter segments based on available criteria and observations.	Multiple segments of different types/statuses exist.	1. Open segment filter UI. 2. Apply a filter (e.g., type/status). 3. Observe map.	Only segments matching the filter are displayed/highlighted; non-matching are hidden or dimmed.	N/A	Skip	Not implemented
System	Web	SEG-003-WEB	Segment filtering	Segments	Verify that user can filter segments based on available criteria and observations.	Multiple segments of different types/statuses exist.	1. Open segment filter UI. 2. Apply a filter (e.g., type/status). 3. Observe map.	Only segments matching the filter are displayed/highlighted; non-matching are hidden or dimmed.	N/A	Skip	Not implemented
System	Mobile	SEG-004-MOBILE	View segment cached data when offline	Segments	Verify that previously loaded segment information can be viewed in offline mode on mobile.	Segment details were opened earlier online on the same device.	1. Open map and select the same segment. 2. Switch mobile device offline. 3. Open segment details.	Segment details are shown from cache; no blocking error; freshness indicator shown if implemented.	User is notified that they are offline. Showing segment details work	Pass	
Integration	Web	USR-001-WEB	User registration	User	Verify that a new user can register to the application.		1. Open registration page. 2. Fill mandatory fields. 3. Submit.	User account is created; success message shown; user can proceed to login.	Register form works requiring all fields	Pass	Phone number missing from backend and frontend forms
Integration	Mobile	USR-001-MOBILE	User registration	User	Verify that a new user can register to the application.		1. Open registration page. 2. Fill mandatory fields. 3. Submit.	User account is created; success message shown; user can proceed to login.	Register form works requiring all fields	Pass	Phone number missing from backend and frontend forms

Integration	Mobile	USR-002-MOBILE	User login	User	Verify that an existing user can log in.	User account exists and is active.	1. Open login page. 2. Enter valid credentials. 3. Submit.	User is authenticated and redirected to main view/dashboard.	Logging in with the correct credentials work	Pass	
Integration	Web	USR-002-WEB	User login	User	Verify that an existing user can log in.	User account exists and is active.	1. Open login page. 2. Enter valid credentials. 3. Submit.	User is authenticated and redirected to main view/dashboard.	Logging in with the correct credentials work	Pass	
Integration	Mobile	USR-003-MOBILE	User logout	User	Verify that a logged-in user can log out.	User is logged in .	1. Open user/profile menu. 2. Select 'Logout'.	Session is terminated; user returned to login screen; protected views no longer accessible.	Logging out works	Pass	
Integration	Web	USR-003-WEB	User logout	User	Verify that a logged-in user can log out.	User is logged in.	1. Open user/profile menu. 2. Select 'Logout'.	Session is terminated; user returned to login screen; protected views no longer accessible.	Logging out of the application works by clicking the same button that is for log in	Pass	
Integration	Mobile	ADM-001-MOBILE	Admin can access admin dashboard	Admin dashboard	Verify that a user with admin role can access the admin dashboard.	Admin user logged in	1. Log in as guide. 2. Navigate to guide dashboard URL/menu entry.	Admin dashboard loads	N/A	Skip	Not implemented
Integration	Web	ADM-001-WEB	Admin can access admin dashboard	Admin dashboard	Verify that a user with admin role can access the dashboard.	Admin user logged in	1. Log in as admin. 2. Navigate to guide dashboard menu entry.	Admin dashboard loads	When logged in, dashboard shows reports, users and segments sections	Pass	
Integration	Mobile	ADM-002-MOBILE	Admin can modify segment coordinates	Admin dashboard	Verify that a admin can update segment coordinate data through the dashboard and it is saved for next loads.	Admin user logged in	1. Open dashboard. 2. Open segment edit view. 3. Change coordinates. 4. Save. 5. Log in / open map as regular user and reload segments.	Segment change is saved in backend; on next load segments reflect the updated coordinates.	N/A	Skip	Not implemented
Integration	Web	ADM-002-WEB	Admin can modify segment coordinates	Admin dashboard	Verify that a admin can update segment coordinate data through the dashboard and it is saved for next loads.	Admin user logged in	1. Open dashboard. 2. Open segment edit view. 3. Change coordinates. 4. Save. 5. Log in / open map as regular user and reload segments.	Segment change is saved in backend; on next load segments reflect the updated coordinates.	N/A	Skip	Not implemented in the backend
Integration	Web	ADM-003-WEB	Admin can modify userdata	Admin dashboard	Verify admin can modify user names, email and roles	Admin user logged in	1. Open dashboard. 2. Open user edit view. 3. Change the details and role of a user. 4. Save. 5. Login with the new user credentials	User change is saved in backend; on next login the user uses those	Admin can modify the fields of other users and change their role	Pass	
Integration	Web	ADM-004-WEB	Admin can create new segments	Admin dashboard	Verify that a admin can update segment coordinate data through the dashboard and it is saved for next loads.	Admin user logged in	1. Open dashboard. 2. Open segment edit view. 3. Create a new segment. 4. Save. 5. Log in / open map as regular user and reload segments.	New segment is created and loaded on the map on next load	N/A	Skip	Not implemented in the backend
Integration	Web	ADM-005-WEB	Admin can create delete segments	Admin dashboard	Verify that a admin can delete segments through the dashboard and it is saved for next loads.	Admin user logged in	1. Open dashboard. 2. Open segment edit view. 3. Delete a segment 4. Save. 5. Log in / open map as regular user and reload segments.	Segment is removed and not shown on the map on next load	N/A	Skip	Not implemented in the backend

Integration	Web	GUD-001-WEB	Guide can add updates to segments	Guide	Verify that a guide can add updates to segments in a similar fashion as normal users but this takes priority	Guide logged in	1. Open guide dashboard. 2. Open map 3. Select segment 4. Add update 5. Reload map	Segment should reflect guide made update on the segment snow conditions	Guide updates on a segment is shown separately	Pass	
Integration	Mobile	GUD-001-MOBILE	Guide can add updates to segments	Guide	Verify that a guide can add updates to segments in a similar fashion as normal users but this takes priority	Guide logged in	1. Open guide dashboard. 2. Open map 3. Select segment 4. Add update 5. Reload map	Segment should reflect guide made update on the segment snow conditions	Guide updates on a segment is shown separately	Fail	When signed in as a guide it still tries to create regular observations (bug list id 3)
Integration	Mobile	OBS-001-MOBILE	Create observation for a segment	Observations	Verify that an observation can be created and associated with a segment.	User logged in; segments available; network available.	1. Open map and select segment. 2. Click 'Add observation'. 3. Fill required fields. 4. Submit.	Observation is saved; visible in segment's information.	Observations can be added to segments	Pass	
Integration	Web	OBS-001-WEB	Create observation for a segment	Observations	Verify that an observation can be created and associated with a segment.	User logged in; segments available; network available.	1. Open map and select segment. 2. Click 'Add observation'. 3. Fill required fields. 4. Submit.	Observation is saved; visible in segment's information.	Observations can be added to segments	Pass	
Integration	Mobile	OBS-002-MOBILE	Load existing observations for segment	Observations	Verify that existing observations can be loaded for a segment.	Segment has at least one observation in backend.	1. Open segment details.	List of observations is shown without errors.	Observations attached to segments are listed on it. Last three and max 3 days old are shown	Pass	
Integration	Web	OBS-002-WEB	Load existing observations for segment	Observations	Verify that existing observations can be loaded for a segment.	Segment has at least one observation in backend.	1. Open segment details.	List of observations is shown without errors.	Observations attached to segments are listed on it. Last three and max 3 days old are shown	Pass	
Integration	Mobile	OBS-003-MOBILE	Guide observations prioritized	Observations	Verify that guide-created observations are prioritized in the observation list view.	Segment has observations by guide and by normal users.	1. Open observations for the segment. 2. Observe order/labeling.	Guide observations appear on top or clearly highlighted according to design.	N/A	Skip	Functionality altered, see test case GUD-006-MOBILE
Integration	Web	OBS-003-WEB	Guide observations prioritized	Observations	Verify that guide-created observations are prioritized in the observation list view.	Segment has observations by guide and by normal users.	1. Open observations for the segment. 2. Observe order/labeling.	Guide observations appear on top or clearly highlighted according to design.	N/A	Skip	Functionality altered, see test case GUD-006-WEB
Integration	Mobile	OBS-004-MOBILE	Observation form flow matches previous application	Observations	Regression check that the observation creation flow in the new app matches the legacy app's flow.	Legacy/reference app available; user logged in.	1. Start observation creation in legacy app and note steps/fields. 2. Start observation creation in new app. 3. Compare step order, fields.	New app follows same flow, fields as legacy app.	Observation can be added with the several different primary snowtypes. Then the flow makes the user select the secondary snow type and can mark hazards	Pass	
Integration	Web	OBS-004-WEB	Observation form flow matches previous application	Observations	Regression check that the observation creation flow in the new app matches the legacy app's flow.	Legacy/reference app available; user logged in.	1. Start observation creation in legacy app and note steps/fields. 2. Start observation creation in new app. 3. Compare step order, fields.	New app follows same flow, fields as legacy app.	Observation can be added with the several different primary snowtypes. Then the flow makes the user select the secondary snow type and can mark hazards	Pass	
Integration	Mobile	OBS-005-MOBILE	Create observation offline	Observations	Verify that observations can be created when offline on mobile and queued for sync.	User logged in earlier; app has required metadata cached.	1. Put device in offline mode. 2. Open segment and create observation. 3. Save. 4. Return online and wait.	Observation is saved locally while offline and marked for later sync; after coming online, observation is sent to backend.	N/A	Skip	Not implemented

System	Mobile	SNW-001-MOBILE	Snowser sensors and data displayed	Snowser	Verify that Snowser sensors appear on the map with their data.	Map loaded; Snowser has sensors.	1. Open map. 2. Enable Snowser layer if disabled. 3. Inspect sensor markers.	All available Snowser sensors are shown on the map with data displayed.	N/A	Skip	Not implemented
System	Web	SNW-001-WEB	Snowser sensors and data displayed	Snowser	Verify that Snowser sensors appear on the map with their data.	Map loaded; Snowser has sensors.	1. Open map. 2. Enable Snowser layer if disabled. 3. Inspect sensor markers.	All available Snowser sensors are shown on the map with data displayed.	Snowser sensors are displayed on the map as dots	Pass	
System	Mobile	SNW-002-MOBILE	Snowser sensor shows temperature and snow depth	Snowser	Verify that temperature and snow depth load for all Snowser sensors in the map.	Snowser sensor exists with data.	1. Open Snowser sensor details. 2. Inspect fields.	Both temperature and snow depth fields are shown; if backend missing a value, UI shows fallback.	N/A	Skip	Not implemented
System	Web	SNW-002-WEB	Snowser sensor shows temperature and snow depth	Snowser	Verify that temperature and snow depth load for all Snowser sensors in the map.	Snowser sensor exists with data.	1. Open Snowser sensor details. 2. Inspect fields.	Both temperature and snow depth fields are shown	Sensor can be clicked to show the details and coordinates	Pass	
System	Mobile	SNW-003-MOBILE	Hide all Snowser sensors from map	Snowser	Verify that all Snowser sensors can be toggled off from the map using layer visibility.	Map loaded; Snowser sensors visible.	1. Open map layers. 2. Toggle Snowser off.	All Snowser markers/data are hidden from map.	N/A	Skip	Not implemented
System	Web	SNW-003-WEB	Hide all Snowser sensors from map	Snowser	Verify that all Snowser sensors can be toggled off from the map using layer visibility.	Map loaded; Snowser sensors visible.	1. Open map layers. 2. Toggle Snowser off.	All Snowser markers/data are hidden from map.	Snowser sensors can be toggled off from the map	Pass	
System	Mobile	SNW-004-MOBILE	Hide Snowser sensor data but keep marker	Snowser	Verify that individual Snowser sensor data (temperature/snow depth label) can be hidden while the sensor remains on the map.	Snowser sensors visible on map.	1. Select a Snowser sensor. 2. Use sensor-specific toggle to hide data. 3. Observe map.	Sensor marker remains visible; its floating data panel is hidden; other sensors are unaffected.	N/A	Skip	Not implemented
System	Web	SNW-004-WEB	Hide Snowser sensor data but keep marker	Snowser	Verify that individual Snowser sensor data (temperature/snow depth label) can be hidden while the sensor remains on the map.	Snowser sensors visible on map.	1. Select a Snowser sensor. 2. Use sensor-specific toggle to hide data. 3. Observe map.	Sensor marker remains visible; its floating data panel is hidden; other sensors are unaffected.	N/A	Skip	Not implemented
System	Mobile	SNW-005-MOBILE	View cached Snowser data offline	Snowser	Verify that mobile users can view last loaded Snowser data when offline.	Snowser data viewed on mobile earlier online.	1. Switch mobile to offline. 2. Open map. 3. Open sensor details.	Last loaded data is displayed from cache; no blocking error.	N/A	Skip	Not implemented
System	Web	SNW-006-WEB	Select snowser sensor to view its details	Snowser	Verify snowser sensors are clickable and their data and coordinates are shown	Snowser sensor exists with data.	1. Open Snowser sensor details. 2. Click a sensor	Both temperature and snow depth fields are shown along the coordinates of the sensor	Snowser sensors display snow depth and temperature	Pass	
Integration	Mobile	RSC-001-MOBILE	Show current coordinates	Rescue	Verify that the mobile app shows the user's current coordinates in the rescue view.	User logged in (mobile); location access granted.	1. Open rescue screen. 2. Observe coordinates shown.	Coordinates are displayed and match device/112 app coordinates (within expected precision).	Rescue view displays user coordinates in the same format as 112 app	Pass	
Integration	Mobile	RSC-002-MOBILE	Start help request with required option	Rescue	Verify that a help request can be created and 1 reason of 3 options is required.	User logged in (mobile); network available.	1. Open rescue screen. 2. Submit help request 3. Select option. 4. Submit.	Help request is created and visible in system.	Help request can be made. User is made to choose reason for help request.	Pass	
Integration	Mobile	RSC-003-MOBILE	Notify if no users nearby	Rescue	Verify that creating a help request when no nearby users exist results in a notification/warning.	User logged in (mobile); backend configured to return no nearby users.	1. Create a help request from an isolated location. 2. Observe response.	User is clearly informed that no users are nearby.	N/A	Skip	Not implemented
Integration	Mobile	RSC-004-MOBILE	Nearby users are notified of help request	Rescue	Verify that nearby users receive both push and in-app notifications about new help requests.	Requester logged in; at least one other mobile user logged in nearby.	1. User A creates help request. 2. On User B's device, observe notifications (push + in-app).	User B gets push notification and in-app alert with request details.	N/A	Skip	Not implemented
Integration	Mobile	RSC-005-MOBILE	Help request can be withdrawn	Rescue	Verify that the requester can cancel/withdraw an active help request.	Active help request exists, created by current user.	1. Open rescue request details. 2. Tap Withdraw/Cancel. 3. Confirm.	Request status becomes cancelled/withdrawn; event is either cancelled or resolved	Help request can be cancelled. User is instructed to select how the event was ended	Pass	

Integration	Mobile	RSC-006-MOBILE	Help request can be accepted	Rescue	Verify that another user can accept an active help request and requester is notified (push + in-app).	Active help request exists; second user logged in on mobile.	1. User B opens a request. 2. User B taps Accept. 3. Observe notifications on requester (User A).	User B is added as responder; User A receives both push and in-app notification.	N/A	Skip	Not implemented
Integration	Mobile	RSC-007-MOBILE	Max 3 users can accept a help request	Rescue	Verify that acceptance is limited to 3 users.	One active help request.	1. Users B, C, D accept request (first three succeed). 2. User E attempts to accept.	First 3 users become responders; 4th user cannot accept and receives appropriate message.	N/A	Skip	Not implemented
Integration	Mobile	RSC-008-MOBILE	All related users can see each other's location and trailing path	Rescue	Verify that users in the same help request can see each other's current locations and path history.	Active help request with requester + at least 1 accepted user.	1. Open map in rescue context as requester. 2. Open map as accepted user. 3. Move one device. 4. Observe locations and trails.	All related users see each other's real-time location markers and recent path/trail.	N/A	Skip	Not implemented
Integration	Mobile	RSC-009-MOBILE	Low battery notification to all related users	Rescue	Verify that all users in the help request receive a notification if the target user's battery is low.	Active help request; target user's device battery status available/simulated as low.	1. Trigger/receive low battery status for target. 2. Observe notifications on all related devices.	All related users receive both push and in-app notifications about low battery.	N/A	Skip	Not implemented
Integration	Mobile	RSC-010-MOBILE	Cancel acceptance of help request	Rescue	Verify that an already accepted user can cancel their acceptance and free a slot.	User has accepted a help request.	1. Open help request. 2. Tap Cancel/Leave. 3. Confirm.	User is removed from responders; available capacity increases; other users still see updated responders list.	N/A	Skip	Not implemented
Integration	Mobile	RSC-011-MOBILE	112 option routes correctly	Rescue	Verify that the 112 option opens dedicated 112 app if installed, otherwise opens dialer with 112.	User logged in (mobile). Two scenarios: device with 112 app; device without 112 app.	1. Open rescue screen. 2. Tap 112. 3. Repeat on device without 112 app.	On device with 112 app: 112 app is opened. On device without 112 app: dialer opens with 112 prefilled.	If the user doesn't have 112 app, they are routed to dial 112, otherwise it redirects to the app	Pass	
Integration	Mobile	RSC-012-MOBILE	Rescue user can access rescue dashboard	Rescue	Verify that a user with Rescue role can access the rescue dashboard on web and mobile.	Rescue-type user exists and logged in.	1. Log in as rescue user. 2. Open rescue dashboard on web. 3. Open rescue dashboard on mobile.	Rescue dashboard loads on both platforms; no access error.	N/A	Skip	Not implemented
Integration	Web	RSC-012-WEB	Rescue user can access rescue dashboard	Rescue	Verify that a user with Rescue role can access the rescue dashboard on web and mobile.	Rescue-type user exists and logged in.	1. Log in as rescue user. 2. Open rescue dashboard on web. 3. Open rescue dashboard on mobile.	Rescue dashboard loads on both platforms; no access error.	N/A	Skip	Not implemented
Integration	Mobile	RSC-013-MOBILE	Search users in rescue dashboard by name or phone	Rescue	Verify that rescue users can search for other users using either name or phone number.	Rescue user logged in; users with known name and phone exist.	1. Open rescue dashboard. 2. Search using full/partial name. 3. Search using phone number.	Matching users are listed for both name and phone queries.	N/A	Skip	Not implemented
Integration	Web	RSC-013-WEB	Search users in rescue dashboard by name or phone	Rescue	Verify that rescue users can search for other users using either name or phone number.	Rescue user logged in; users with known name and phone exist.	1. Open rescue dashboard. 2. Search using full/partial name. 3. Search using phone number.	Matching users are listed for both name and phone queries.	N/A	Skip	Not implemented
Integration	Mobile	RSC-014-MOBILE	Show last 5 locations with timestamp in rescue dashboard	Rescue	Verify that rescue users can view the last 5 locations of a selected user, with timestamps, in correct order.	Rescue user logged in; target user has at least 5 location entries.	1. Open rescue dashboard. 2. Open a user's details. 3. Open location history section.	Exactly 5 latest locations are shown, each with timestamp, ordered newest to oldest.	N/A	Skip	Not implemented
Integration	Web	RSC-014-WEB	Show last 5 locations with timestamp in rescue dashboard	Rescue	Verify that rescue users can view the last 5 locations of a selected user, with timestamps, in correct order.	Rescue user logged in; target user has at least 5 location entries.	1. Open rescue dashboard. 2. Open a user's details. 3. Open location history section.	Exactly 5 latest locations are shown, each with timestamp, ordered newest to oldest.	N/A	Skip	Not implemented

Integration	Mobile	WTH-001-MOBILE	View weather for day before yesterday, yesterday, today	Weather	Verify that user can view weather for the last 3 days and that values correspond to FMI data.	Weather data for D-2, D-1, and D available in the FMI	1. Open weather view. 2. Navigate to day before yesterday, yesterday, and today.	Weather data is shown for each of the 3 days; values match the api data.	N/A	Skip	Feature rejected
Integration	Web	WTH-001-WEB	View weather for day before yesterday, yesterday, today	Weather	Verify that user can view weather for the last 3 days and that values correspond to FMI data.	Weather data for D-2, D-1, and D available in the FMI	1. Open weather view. 2. Scroll/browse to previous days as allowed by UI.	Weather data is shown for each of the 3 days; values match the api data.	N/A	Skip	Feature rejected
System	Mobile	WTH-002-MOBILE	View weather for last few days	Weather	Verify that user can browse historical weather as defined by the rework showcase.	Reworked weather UI available	1. Open weather view. 2. Scroll/browse to previous days as allowed by UI.	Historical days are displayed according to UI limits; each day shows data; exact content validated by unit tests.	N/A	Skip	Feature rejected
System	Web	WTH-002-WEB	View weather for last few days	Weather	Verify that user can browse historical weather as defined by the rework showcase.	Reworked weather UI available	1. Open weather view. 2. Scroll/browse to previous days as allowed by UI.	Historical days are displayed according to UI limits; each day shows data; exact content validated by unit tests.	N/A	Skip	Feature rejected
System	Mobile	WTH-003-MOBILE	View wintertime weather observations	Weather	Verify that user can view wintertime-specific weather observations.	Wintertime data present	1. Open weather view. 2. Select wintertime/winter observations section.	Wintertime weather observations are displayed without errors and content is correct.	N/A	Skip	Feature rejected
System	Web	WTH-003-WEB	View wintertime weather observations	Weather	Verify that user can view wintertime-specific weather observations.	Wintertime data present	1. Open weather view. 2. Select wintertime/winter observations section.	Wintertime weather observations are displayed without errors and content is correct.	N/A	Skip	Feature rejected
System	Mobile	WTH-004-MOBILE	View cached weather when offline	Weather	Verify that when offline on mobile, the user can view the latest loaded weather information from cache.	User opened weather view earlier online on the same device.	1. Switch mobile device offline. 2. Open weather view.	Latest loaded weather info is displayed from cache; user is not blocked by missing connection.	N/A	Skip	Feature rejected
Integration	Web	WTH-005-WEB	Reworked weather view	Weather	Verify that user can view weather page that includes; Average wind of last 3 days Average temperature of last 3 days Max wind speed of last 3 days Snow depth change of last 7 days How many of last 3 days were above freezing		1. Open weather view.	The listed weather details is displayed	Weather view displays avg wind and temp of 3 days, max wind of last 3 days, snow depth change in 7 days and how many days out of last 3 were above freezing	Pass	
Integration	Mobile	WTH-005-MOBILE	Reworked weather view	Weather	Verify that user can view weather page that includes; Average wind of last 3 days Average temperature of last 3 days Max wind speed of last 3 days Snow depth change of last 7 days How many of last 3 days were above freezing		1. Open weather view.	The listed weather details is displayed	Weather view displays avg wind and temp of 3 days, max wind of last 3 days, snow depth change in 7 days and how many days out of last 3 were above freezing	Pass	

APPENDIX B: Table of Found Bugs and Issues

Platform	ID	Description	Will be fixed	Fixed	Verified
all	1	cannot input phone number when registering	n	n	n
mobile	2	centering shortcut button working inconsistently			
mobile	3	guide updates not working on guide or admin accounts, creates only regular snow observations	n	n	n
mobile	4	info text weather remove "the entire season"			
mobile	5	map centering on location	y	y	y
mobile	6	map on airplane mode	y	y	y
mobile	7	map scale and compass overlap the phone status bar	y	y	y
mobile	8	moving between locations should make the button greyed out until the position is moved there	n	n	n
mobile	9	replace rescues with wanderer in rescue view	y	y	y
mobile	10	rescue button to red colour	y	y	y
mobile	11	Rescue end point not connected to backend	y	y	y
mobile	12	rescue page not scrollable/scalable	y	y	y
mobile	13	segment and sensor filter obstruct the bottom navbar	n	n	n
mobile	14	segment filtering button color (grey/green as it does on web)	n	n	n

Selecting outer segments will also					
mobile	15	highlight their inner segments	n	n	n
mobile	16	set landing page to map	y	y	y
mobile	17	severe mergency only 112	y	y	y
shows observations over 3 days					
mobile	18	old	n	n	n
snow type names and desc always shown in finnish on observations					
mobile	19		n	n	n
mobile	20	stones and branches translation	n	n	n
User should not be able to send help request when logged out or offline					
mobile	21				
does not show 3 day old observations					
web	22				
info correction					
web	23				
info text weather remove "the entire season"					
web	24		y	y	y
web	25	login button help text	y	y	y
make snowser sensor size larger on map					
web	26		y	y	y
map menu toggle button hidden behind nav on mobile					
web	27		y	y	y
primary & secondary snowtypes text is only in english when creating guide updates as guide or admin					
web	28		y	y	y
report button shown when not logged in					
web	29		y	y	y

web	30	Selecting outer segments should not highlight their inner segments	y	y	y
web	31	unable to make lower segments	n	n	n
web	32	weather rounding	y	y	y
web	33	map takes long to load	y	y	y
mobile	34	missing pallaksen pöllöt info text	n	n	n

APPENDIX C: Acceptance Testing Results and Response

Test Report

Juuso Holstein & Arto Hämäläinen

Later comments by G05

Mobile Version

Bugs

- After visiting Snow definitions, there is no way to get back - “dead end”
 - A back button will be added to the page.
- Snow depth information is missing in the weather section. Also other data as temperature within the last 3 days is not there, as well as the wind (number of days above 20 m/s)
 - To us it seemed like this was well specified, since the weather page content was specifically discussed and agreed upon. Regretfully this was not the case, will be marked in “Future development” as we really do not have the time to fix this.
- Snow detectors not available
 - This was left out of scope due to time constraints. This was also mentioned by us in a meeting.
- Snow segment information goes out of screen on iphone. Headline is under the camera box.
 - There were some scaling issues that were fixed. We tested this on the newest version on the iphones available to us and could not reproduce the issue.
- Shows segments (the button in the left hand corner), but as in the previous version, there were snow types that you are able to choose from and it shows all the segment areas color coded -> this is missing totally?
 - This was honestly a mistake on management part and the implementation of the snow type segment filtering was overlooked and a bit forgotten even though it was specified as a work item. About the color coding, it was a bit of a challenge to find a smart way to implement this and it likely would have involved changes in backend which were not the fastest to get. Marking both for future development.

- When people start using the app, there should be the log-in! In addition as we talked earlier, the identification should go by phone number, now there is no such thing at all!
 - Login not being there on the application launch was something that was quite plainly overlooked regretfully. No resources to fix this at this point, marking it as future improvements.
 - Phone number was once again one thing that was not added in the backend registration and therefore it was left out on the frontend registration. Also there should be a way to validate a phone number when user tries to login in or register using one, and implementing that was out of scope for this project. This is marked as future improvements.
- Rescue page too big for the screen!
 - Rescue page had some scaling issues in the first version that were addressed and fixed.

Challenges / User experience

- User page as a login page
 - We interpreted this to mean that the issue is that login page is a bit “hidden” from the actual user of the application. Marking this as future improvements.
- Rescue button painted red
 - This will be fixed and colour changed to red.
- Rescue feature
 - Help -request alerts and sends your location to nearby wanderer, not rescuer
 - We presume that this addresses the text there, rescuer will be changed to wanderer
 - In a severe emergency, always call 112

- We presume that this also addresses the text, will add modify it to “severe emergency”
- Snow segment information is hard to follow as there are so many different listings -> only the most recent ones should be shown
 - This is fixed in a later version of the application.
- I would say the rescue button (as it was in the previous version), could be implemented to the banner, so that it is in the middle and highlighted
 - Marking this as a future improvement, not feasible to do with the hours left.

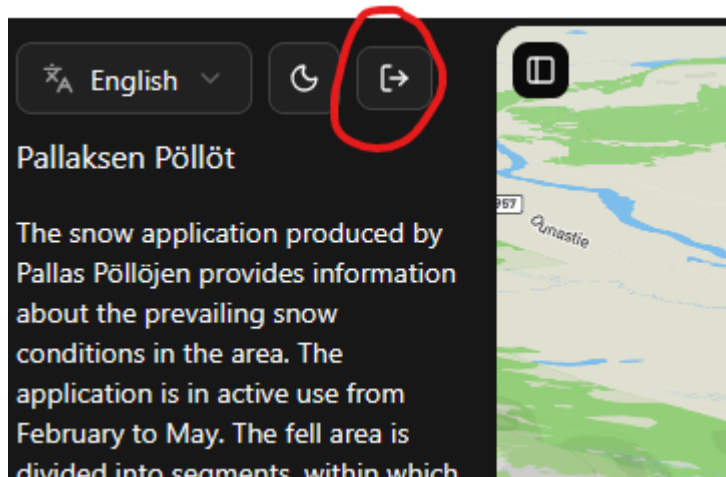
Future developments

- More detailed map
 - If by this you mean how the map looks, mapbox offers more map styles to choose from. Marking this as future ideas, this be easy to change in the future.

Browser Version

Bugs

- You can't add observations, lack of verbal feed back
 - Observations cannot be made when you are not logged in. There was an issue where the option to do this was left there. This is fixed in a new version.
- How to log in?
 - There is a login button present. Text will be added there to make it clearer.



-
- Location?
 - Unsure what is exactly meant by this. Location cannot really be shown on the web application unless a phone is used. We thought that the intention was to use the mobile app on a phone, not the web app via browser.
- The most recent observations hard to find -> there are no visual symbols or color coding as it was in the previous version
 - We assume that this colour issue references to the same one as earlier.
 - Unsure what is mean by the most recent observation, application shows the most recent observations, oldest being max 3 days old.
- Weather site has incorrect wind reports (too many decimals)
 - This was identified and fixed in a newer version.
- There are even less weather information than in the application -> reason for that?
 - Both of the applications contain identical weather data. It is just represented a bit differently, which could/should be unified, marking that for future improvements.

Challanges / User experience

- Definitions are taking too vast role, could be within observations. Perhaps using the left hand section were now is "Pallaksen Pöllöt"
 - The style for showing definitions was adapted from the previous application, where they are contained on their own page. Marking this as future improvements.

- It takes quite long to load the map everytime you move around the site
 - Made an adjustment related to this on a newer version, where it now is faster.
- Snow sensor is visible here - will it be added automatically once we add new meters to the system?
 - New sensors that are available in the snowier are automatically added here. There should not be any issues with this
- In addition, i would say the mark on the map is challenging especially when in the crossroads of segments
 - We interpreted this being an issue with the size of the snowier sensor dot on the map. We made it a bit bigger.

Future developments

- Could we follow friends here
 - Unsure what this exactly mean, should be marked for future development once clarified.